Imports

1. First, import all the necessary modules using the import function. For this exercise, we will be mainly using `pandas`. To learn more about these packages, you can read through the documentation:
   - https://pandas.pydata.org/
   - https://numpy.org/
   - https://seaborn.pydata.org/
   - https://matplotlib.org/

```
# Let's import the relevant Python packages here
# Feel free to import any other packages for this project

#Data Wrangling
import numpy as np
import pandas as pd

#Plotting
import matplotlib as mpl
import matplotlib.pyplot as plt
import seaborn as sns

%matplotlib inline
```

Data

In the following exercise, we will familiarize ourselves with Python by studying the College dataset, which can be found in the file `College.csv`. This dataset contains the following variables from 777 different universities and colleges in the US:

Column	Description
Private	Public/private indicator
Apps	Number of applications received
Accept	Number of applicants accepted
Enroll	Number of new students enrolled
Top10perc	New students from top 10% of high school class
Top25perc	New students from top 25% of high school class
F.Undergrad	Number of full-time undergraduates
P.Undergrad	Number of part-time undergraduates
Outstate	Out-of-state tuition
Room.Board	Room and board costs
Books	Estimated book costs
Personal	Estimated personal spending
PhD	Percent of faculty with Ph.D.'s
Terminal	Percent of faculty with terminal degree
S.F.Ratio	Student/faculty ratio
Perc.alumni	Percent of alumni who donate
Expend	Instructional expenditure per student
Grad.Rate	Graduation rate

2. Load the College dataset using `pandas`

```
###code here
college = pd.read_csv('/content/sample_data/College.csv')
```

3. Use the `head()` function to view the data.

```
###code here
college.head()
```

	Private	Apps	Accept	Enroll	Top10perc	Top25perc	F.Undergrad	P.Undergrad	Outstate	Room.Board	Books	Personal
0	Yes	1660	1232	721	23	52	2885	537	7440	3300	450	2200
1	Yes	2186	1924	512	16	29	2683	1227	12280	6450	750	1500
2	Yes	1428	1097	336	22	50	1036	99	11250	3750	400	1165
3	Yes	417	349	137	60	89	510	63	12960	5450	450	875
4	Yes	193	146	55	16	44	249	869	7560	4120	800	1500

Next steps: Generate code with college | New interactive sheet

4. Notice that there is a column 'Names' of each university's name. As we don't want to use these names as predictors, they are natural candidates to index our data.

```
###code here
college.index
```

```
RangeIndex(start=0, stop=777, step=1)
```

5. Use the `head()` function again. You should now see that the indices have been replaced with the name of each university in the data set. This means that Python has given each row a name corresponding to the appropriate university. Python will not try to perform calculations on the row names.

```
###code here
college.head()
```

	Private	Apps	Accept	Enroll	Top10perc	Top25perc	F.Undergrad	P.Undergrad	Outstate	Room.Board	Books	Personal
0	Yes	1660	1232	721	23	52	2885	537	7440	3300	450	2200
1	Yes	2186	1924	512	16	29	2683	1227	12280	6450	750	1500
2	Yes	1428	1097	336	22	50	1036	99	11250	3750	400	1165
3	Yes	417	349	137	60	89	510	63	12960	5450	450	875
4	Yes	193	146	55	16	44	249	869	7560	4120	800	1500

Next steps: Generate code with college | New interactive sheet

6. Use the `info()` function to check and produce a numerical summary of your variables.

```
###code here
college.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 777 entries, 0 to 776
Data columns (total 19 columns):
 #   Column       Non-Null Count  Dtype
---  ------       --------------  -----
 0   Private      777 non-null    object
 1   Apps         777 non-null    int64
 2   Accept       777 non-null    int64
 3   Enroll       777 non-null    int64
 4   Top10perc    777 non-null    int64
 5   Top25perc    777 non-null    int64
 6   F.Undergrad  777 non-null    int64
 7   P.Undergrad  777 non-null    int64
```

```
 8   Outstate     777 non-null    int64
 9   Room.Board   777 non-null    int64
 10  Books        777 non-null    int64
 11  Personal     777 non-null    int64
 12  PhD          777 non-null    int64
 13  Terminal     777 non-null    int64
 14  S.F.Ratio    777 non-null    float64
 15  perc.alumni  777 non-null    int64
 16  Expend       777 non-null    int64
 17  Grad.Rate    777 non-null    int64
 18  Names        777 non-null    object
dtypes: float64(1), int64(16), object(2)
memory usage: 115.5+ KB
```

7. Examine if there are any duplicates and drop them if needed.

```
###code here
college.duplicated().sum()
```

```
np.int64(0)
```

8. Replace any missing values in the `Apps' column with 0. This dataframe will henceforth be our original dataframe.

```
###code here
college.Apps.fillna(0,inplace = True)
```

```
/tmp/ipython-input-3976503871.py:2: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series thr
The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or d


  college.Apps.fillna(0,inplace = True)
```

9. Find the college with the least out-of-state tuition and name this variable `college_least_tuition`. The variable should return the name of a college, not its tuition. **Q:Which college in this dataset has the least amount of out-of-state tuition?**

```
###code here
#method 1

min_tuition = college.Outstate.min()
college[college.Outstate == min_tuition]

# Method 2
college.Outstate.idxmin()

'Brigham Young University at Provo'
```

```
'Brigham Young University at Provo'
```

10. From the original dataframe, select the 'PhD' column and name this dataframe `phd_column`. Find the length of this column and name this variable `phd_column_length`.

```
# Note: Double bracket for dataframe
phd_column = college[['PhD']]
```

11. From the original dataframe, select both the 'Private' and 'Top10perc' columns, and slice them such that only the rows with index 15 and 16 remain. Name this dataframe `private_top10`. From this dataframe, find the length of the filtered 'Private' column and name this variable `private_column_length`.

```
# Double bracket to obtain multiple columns
private_column_length = college[['Private', 'Top10perc']].iloc[15:17]
```

12. From the original dataframe, select the row that only contains data about the "Amherst College". Name this dataframe `Amherst`.

```
###code here
Amherst = college[college.index == 'Amherst College']
```

13. From the original dataframe, select the rows that contain all colleges with the name "Penn" included. The "P" should be capitalized. Name this dataframe `many_penns`. Comment on your observations. **Q: How many universities are there in this dataset that include "Penn"?**

```
###code here
many_penns = college[college['Names'].str.contains('Penn')]

print(len(many_penns), "universities are there in this dataset that include 'Penn'")
```

```
8 universities are there in this dataset that include 'Penn'
```

Exploratory Data Analysis (EDA)

we will perform exploratory data analysis (EDA) to extract insights

First, we return to the College dataset. This dataset contains the following variables from 777 different universities and colleges in the US:

Column	Description
Private	Public/private indicator
Apps	Number of applications received
Accept	Number of applicants accepted
Enroll	Number of new students enrolled
Top10perc	New students from top 10% of high school class
Top25perc	New students from top 25% of high school class
F.Undergrad	Number of full-time undergraduates
P.Undergrad	Number of part-time undergraduates
Outstate	Out-of-state tuition
Room.Board	Room and board costs
Books	Estimated book costs
Personal	Estimated personal spending
PhD	Percent of faculty with Ph.D.'s
Terminal	Percent of faculty with terminal degree
S.F.Ratio	Student/faculty ratio
Perc.alumni	Percent of alumni who donate
Expend	Instructional expenditure per student
Grad.Rate	Graduation rate

First, Let's rename some columns.

```
college = college.rename(columns = {'Grad.Rate':'Grad_Rate',
                                    'S.F.Ratio': 'S_F_Ratio',
                                    'perc.alumni':'perc_alumni',
                                    'Room.Board':'Room_Board',
                                    'F.Undergrad':'F_Undergrad',
                                    'P.Undergrad':'P_Undergrad'})
```

```
college.Grad_Rate
```

	Grad_Rate
0	60
1	56
2	54
3	59
4	15
...	...
772	40
773	83
774	49
775	99
776	99

777 rows × 1 columns

dtype: int64

```
college.columns = college.columns.str.lower()
```

1. Use the `seaborn` `pairplot()` function on the original dataframe with all universities to produce a scatterplot matrix of the first ten columns of the data. Remember to use the `%matplotlib` inline command for plotting in Jupyter. Comment on your observations.

```
###code here
college10 = college.iloc[:,:10]
sns.pairplot(college10)
plt.show()
```

These plot shows strong positive relationships between applications,acceptances, and enrollments. Top10perc and Top25perc are positively correlated. Most size-related variables are right-skewed, indicating that only a few schools are very large. These plots suggest that school size and student quality are closely related.

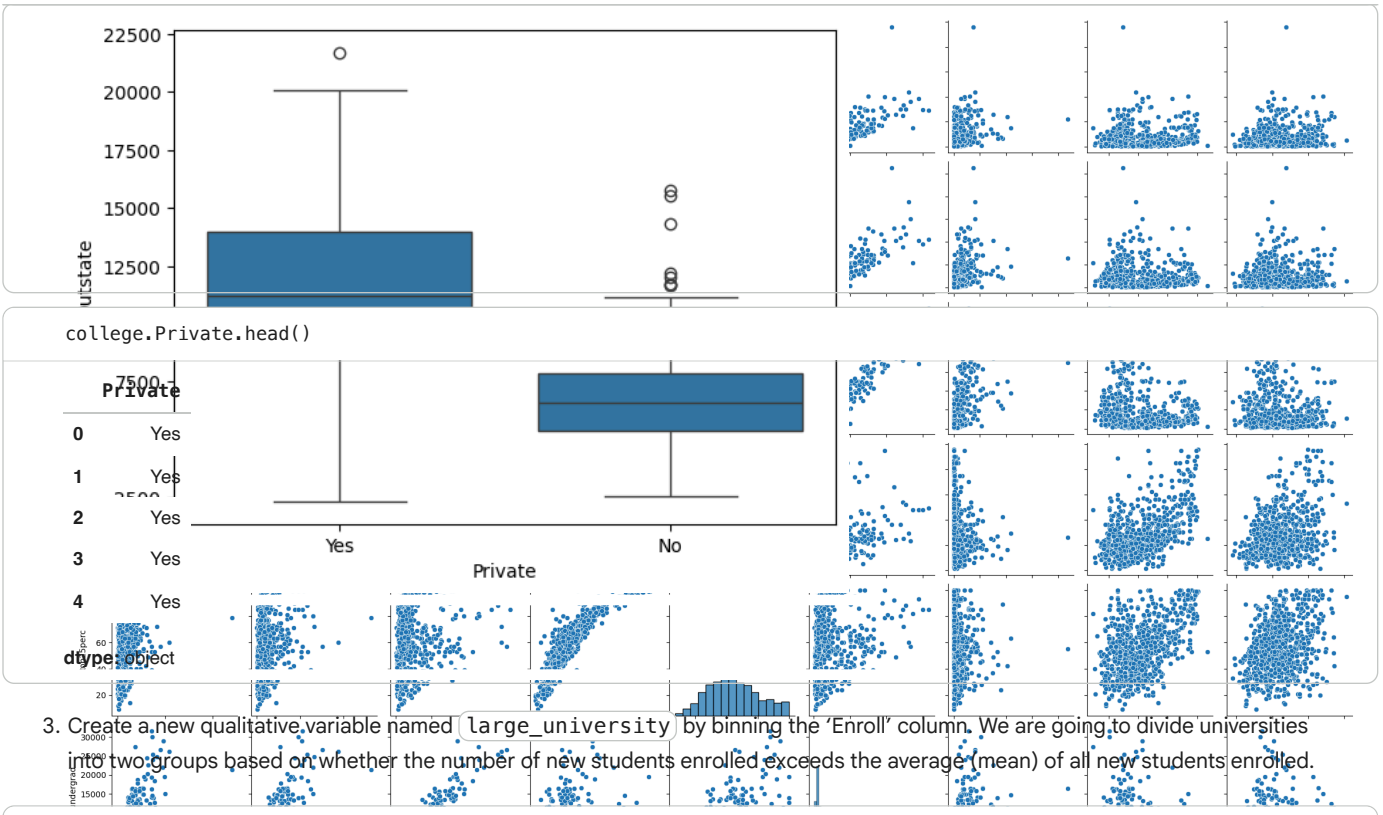
2. Use the `seaborn` `boxplot()` function on the original dataframe with all universities to produce side-by-side box plots of 'Outstate' versus 'Private'. **Q: On average, are private universities more expensive than public universities?**

```
###code here

college = pd.read_csv('/content/sample_data/College.csv')

sns.boxplot(x='Private', y='Outstate', data=college)
plt.show()

#Answer: Yes,private universities are more expensive than public universities on average.
```




```
college.Private.head()
```

	Private
0	Yes
1	Yes
2	Yes
3	Yes
4	Yes

dtype: object

3. Create a new qualitative variable named `large_university` by binning the 'Enroll' column. We are going to divide universities into two groups based on whether the number of new students enrolled exceeds the average (mean) of all new students enrolled.

```
###code here
avg_enroll = college.Enroll.mean()
avg_enroll
```

```
np.float64(779.9729729729729)
```

```
large_university = college['Enroll'].apply(lambda x: True if x > avg_enroll else False)
```

```
large_university
```

	Enroll
0	False
1	False
2	False
3	False
4	False
...	...
772	False
773	False
774	False
775	True
776	False

777 rows × 1 columns

dtype: bool

4. Create a dataframe called `large_universities` that only includes large universities.

```
###code here
large_universities = college[large_university == True]
```

```
len(large_universities)
```

```
218
```

5. Use the `pandas` `describe(include = 'all')` function on large_universities to produce a numerical summary of each column. Within this dataframe, what is the 75th percentile for the column 'Enroll'?

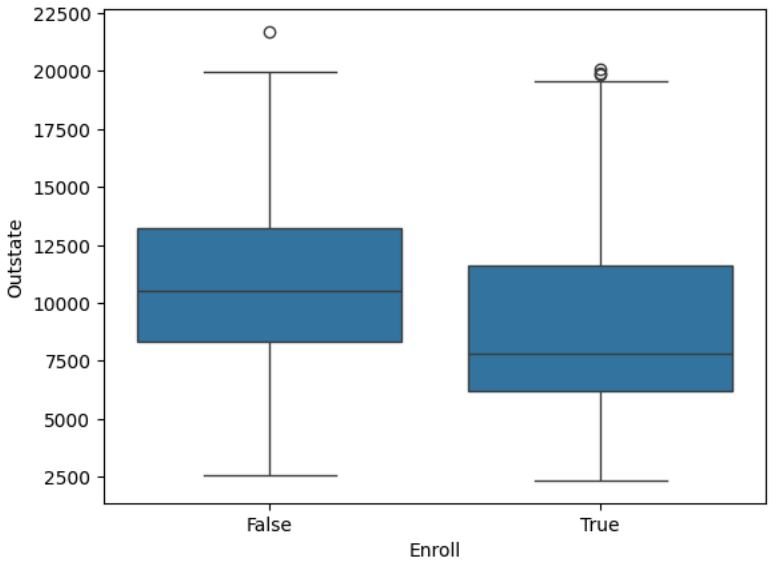
```
###code here
large_universities.describe(include = 'all')
```

	Private	Apps	Accept	Enroll	Top10perc	Top25perc	F.Undergrad	P.Undergrad	Outstate	Room.I
count	218	218.000000	218.000000	218.000000	218.000000	218.000000	218.000000	218.000000	218.000000	218.0
unique	2	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	
top	No	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	
freq	151	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	
mean	NaN	7233.376147	4734.110092	1897.229358	31.224771	60.298165	9455.123853	1995.334862	9258.600917	4311.8
std	NaN	5069.744625	3186.820612	1124.121061	21.994342	21.117579	5978.291505	2369.873736	4409.338645	1246.8
min	NaN	1256.000000	1068.000000	781.000000	3.000000	18.000000	1320.000000	16.000000	2340.000000	1780.0
25%	NaN	3943.250000	2686.500000	1058.000000	15.000000	45.000000	5023.250000	658.500000	6197.250000	3413.0
50%	NaN	6003.500000	3752.500000	1548.000000	25.000000	57.500000	7533.500000	1354.500000	7822.000000	4077.5
75%	NaN	8756.500000	5692.250000	2408.000000	40.000000	74.500000	12791.750000	2574.000000	11596.000000	5085.5
max	NaN	48094.000000	26330.000000	6392.000000	96.000000	100.000000	31643.000000	21836.000000	20100.000000	7425.0

6. Use the `seaborn` `boxplot()` function on the large_university variable to determine whether larger universities are more expensive. **Q: are large universities more expensive than small universities?**

```
###code here
sns.boxplot(x = large_university,y = college.Outstate)
plt.show()

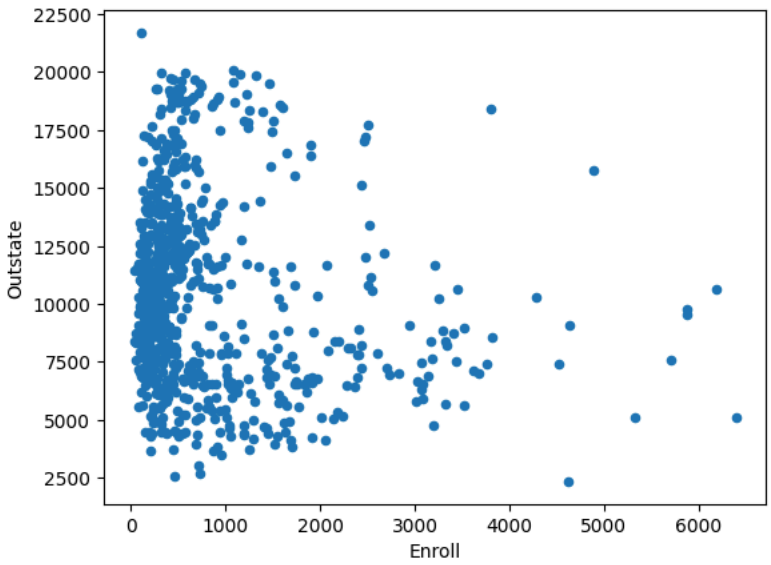
#Answer: large universities are not more expensive thatn small universities
```



7. To further examine the relationship between the size of a university against its out-of-state tuition, plot a scatter plot. **Q: What type of relationship does this show (weak, medium, strong)?**

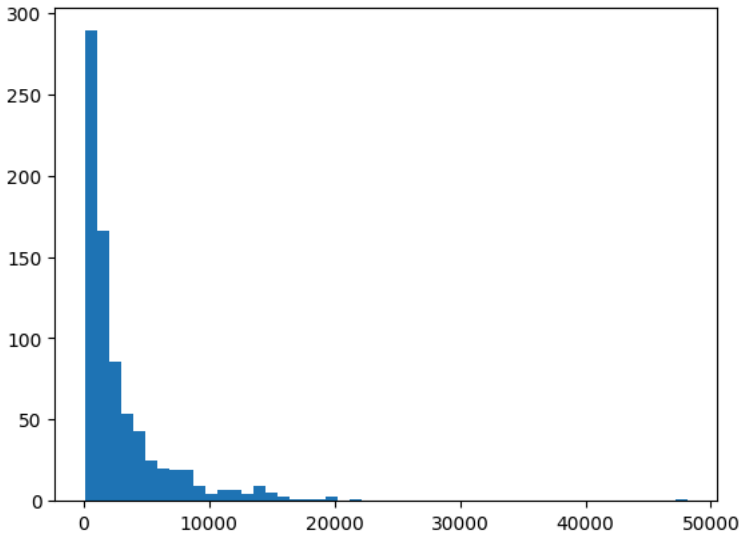
```
###code here
college.plot.scatter('Enroll', 'Outstate')
plt.show()
```

```
#Answer: The plot shows a weak correlation between size of university & Out of state tuition
```



8. Use `matplotlib.pyplot.hist()` or `seaborn.distplot()` to produce some histograms with different numbers of bins for a few of the quantitative variables. You may find the `matplotlib.pyplot.subplot()` function useful to plot multiple graphs under a single codeblock. In particular, create a histogram to examine the number of applications received with a bin size of 200. **Q: Are there more universities that receive 0 to 10,000 applications or 10,000 to 20,000 applications?**

```
###code here
plt.hist(college.Apps,bins = 50)
plt.show()
```

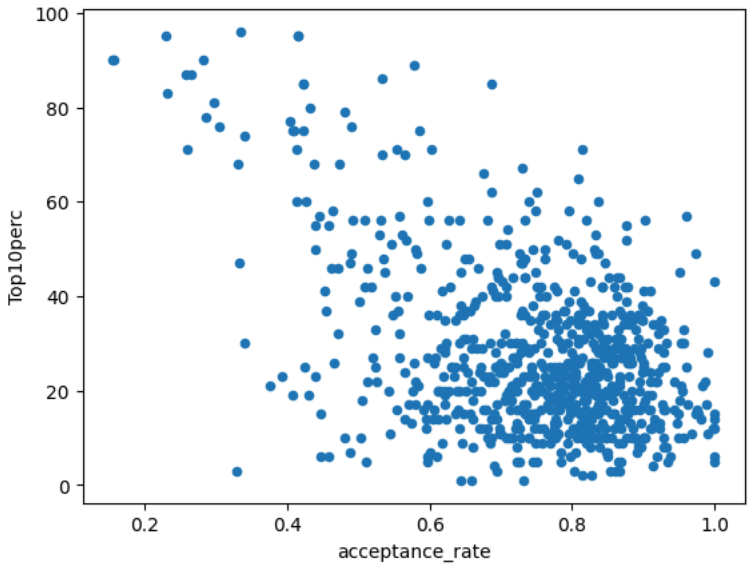


9. We are interested in the acceptance rate of these universities. To calculate the acceptance rate, we take the 'Accept' column and divide by the 'Apps' column. Add this column to a copied version of the original dataframe. Remember to copy the dataframe first before performing feature transformation, as we do not wish to alter or inadvertently create new columns in our original data. **Name your new column `acceptance_rate`. Q: Is there a positive or negative relationship between the university's acceptance rate and the percentage of students in the university who graduated from the top 10% from high school?**

```
###code here
college['acceptance_rate'] = college.Accept/college.Apps
```

```
college.plot.scatter('acceptance_rate', 'Top10perc')
plt.show()

#Answer: There is a negative realtionship between acceptance rate& top10perc
```




```
college[['acceptance_rate', 'Top10perc']].corr()
```

	acceptance_rate	Top10perc
acceptance_rate	1.000000	-0.478675
Top10perc	-0.478675	1.000000

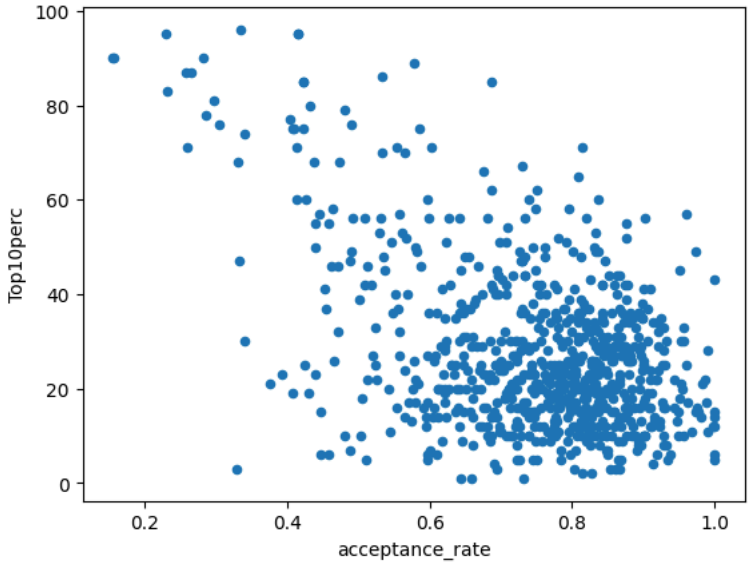
You may plot the university's acceptance rate vs the percentage of students in the university who graduated from the top 10% from high school.

```
###Code here
college_copy = college.copy()

college_copy['acceptance_rate'] = college_copy.Accept / college_copy.Apps

college_copy.plot.scatter('acceptance_rate', 'Top10perc')
```

```
<Axes: xlabel='acceptance_rate', ylabel='Top10perc'>
```



10. Continue exploring the data and provide a brief summary of what you discover.

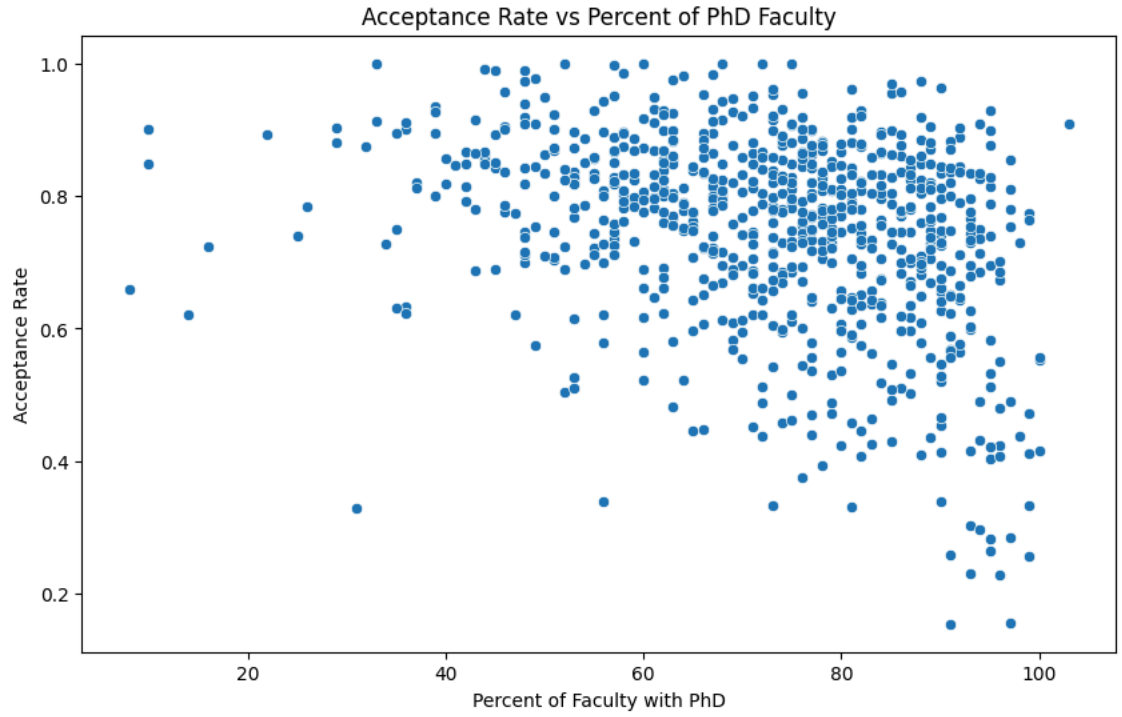
Research question 1: Whether universities with higher percent of PhD faculties has a higher acceptance rate?

```
###Code here
import seaborn as sns
import matplotlib.pyplot as plt
import pandas as pd

college = pd.read_csv('/content/sample_data/College.csv')
college['acceptance_rate'] = college['Accept'] / college['Apps']

plt.figure(figsize=(10, 6))
sns.scatterplot(x='PhD', y='acceptance_rate', data=college)
plt.title('Acceptance Rate vs Percent of PhD Faculty')
plt.xlabel('Percent of Faculty with PhD')
plt.ylabel('Acceptance Rate')
plt.show()

print("Correlation:")
print(college[['PhD', 'acceptance_rate']].corr())
```




```
Correlation:
                     PhD  acceptance_rate
PhD             1.000000        -0.318334
acceptance_rate -0.318334         1.000000
```

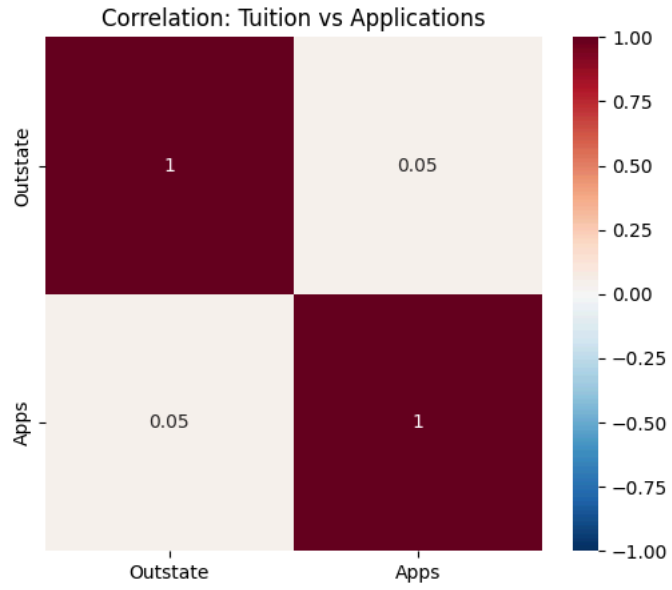
The plot and analysis show only a weak negative correlation between the percentage of faculty holding PhD degrees and acceptance rates suggesting that universities with higher proportions of PhD faculty tend to be slightly more selective.

Research question 2: Will higher tuition cost have negative impact on application numbers?

```
corr = college[['Outstate', 'Apps']].corr()

plt.figure(figsize=(6,5))
sns.heatmap(corr, annot=True, cmap='RdBu_r', center=0, vmin=-1, vmax=1)
plt.title('Correlation: Tuition vs Applications')
plt.show()

corr
```



Next steps: Generate code with corr New interactive sheet

```
            Outstate      Apps
Outstate    1.000000  0.050159
```

```
correlation = college[['Outstate', 'Apps']].corr().iloc[0, 1]
print(f"Correlation between Tuition and Applications: {correlation:.4f}")
if correlation < 0:
    print("Discovery: There is a negative correlation, suggesting higher tuition might deter applicants.")
else:
    print("Discovery: There is a positive correlation, suggesting that more expensive colleges actually attract more
```

```
Correlation between Tuition and Applications: 0.0502
Discovery: There is a positive correlation, suggesting that more expensive colleges actually attract more application
```

Correlation between Tuition and Applications: 0.0502. There is a positive correlation, but it shows a relatively weak correlation rather than a strong positive correlation. This suggests that higher tuition cost does increase the number of applications slightly.

Summary:

The analysis for both research questions revealed weak correlations between the variables. PhD faculty percentage showed weak to moderate association with acceptance rates, while tuition costs demonstrated similarly limited predictive power for application numbers. This pattern suggests that university selectivity and admissions operate through more complex mechanisms than these institutional characters alone can explain. The weak correlations reflect the multidimensional nature of university reputation, and factors such as graduation outcomes, alumni donations, resource allocation, and student-faculty ratios likely interact in ways that single-variable analysis cannot capture.